

Modelação e Design

15: Diagrama de Classes

Leonor Melo
leonor@isec.pt

1

Diagrama de classes

- Classes Tipo de Dados
- Classes abstratas
- Interfaces
- Templates

Leonor Melo

15 - Diagramas de classe

2

2

Tipo dos atributos - 1

- Tipo dos atributos de uma classe
 - Sobretudo tipos de dados "primitivos"
 - números,
 - valores booleanos,
 - carater,
 - strings,..

Leonor Melo

og Modelo do Domínio

3

3

Tipo dos atributos - 2

- Tipo dos atributos (continuação)
 - Mas tb podem ser "tipos de dados" comuns
 - Endereço,
 - Número de Telefone,
 - Código Postal,
 - URL,
 - Cor,
 - Figura Geométrica,...

Leonor Melo

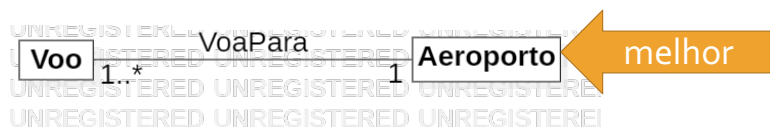
og Modelo do Domínio

4

4

Tipo dos atributos - 3

- Conceitos complexos
 - classes conceptuais



Leonor Melo

og Modelo do Domínio

5

5

Classe tipo de dados ou classe conceptual? - 1

- “Tipos de dados” comuns
 - conjuntos de valores para os quais
 - uma identidade única não é relevante
 - no nosso modelo
 - teste de igualdade
 - baseiam-se no valor
 - não na identidade

Leonor Melo

og Modelo do Domínio

6

6

Classe tipo de dados ou classe conceptual? –2

- Exemplo:
- Não faz sentido distinguir:
 - instancias separadas do valor inteiro 5
 - instancias separadas da string "Bom dia"
 - instancias separadas da data "13 de abril 2016"
- Faz sentido distinguir
 - instancias separadas de *Pessoa* mesmo se por acaso tem ambas o nome "João Silva"

Leonor Melo

og Modelo do Domínio

7

7

Classe tipo de dados ou classe conceptual? - 3

- Do ponto de vista do software:
 - comparamos dois inteiros ou duas datas
 - por valor
 - comparamos duas Pessoas
 - usando o endereço de memória:
 - é possível duas pessoas com
 - os mesmos atributos
 - mas identidades diferentes

Leonor Melo

og Modelo do Domínio

8

8

Classe tipo de dados ou classe conceptual? - 4

- Tipos de dados
 - (usualmente) imutáveis ao longo do tempo:
 - a instancia do inteiro 5
 - não muda
 - a instancia da data "13 de abril 2016"
 - provavelmente não mudará
 - uma Pessoa
 - pode mudar de nome

Leonor Melo

o9 Modelo do Domínio

9

9

Classe tipo de dados ou classe conceptual? - 3

- Classe conceptual
 - composta por várias secções
 - secções usadas individualmente
 - nome,
 - número de telefone
 - Indicativo de país,
 - Indicativo região / operadora,
 - número
 - número BI
 - 7/8 algarismos
 - 1º dígito de controlo
 - 2 letras (número de emissões)
 - 2º dígito de controlo

Leonor Melo

o9 Modelo do Domínio

10

10

Classe tipo de dados ou classe conceptual? - 3

- Classe conceptual (continuação):
 - Tem outros atributos
 - preço promocional,
 - valor,
 - data de início,
 - data de fim
 - Quantidade com uma unidade associada
 - pagamento
 - valor numérico,
 - moeda
 - medida
 - valor,
 - unidade

Leonor Melo

o9 Modelo do Domínio

11

11

Classe tipo de dados ou classe conceptual? - 4

- Classes tipo de dados
 - Representadas explicitamente no modelo do domínio
 - relevância de conhecer os atributos dessa classe para
 - representar,
 - compreender, e
 - comunicar
 - sobre o domínio
 - Não representadas explicitamente
 - tipo de determinado atributo

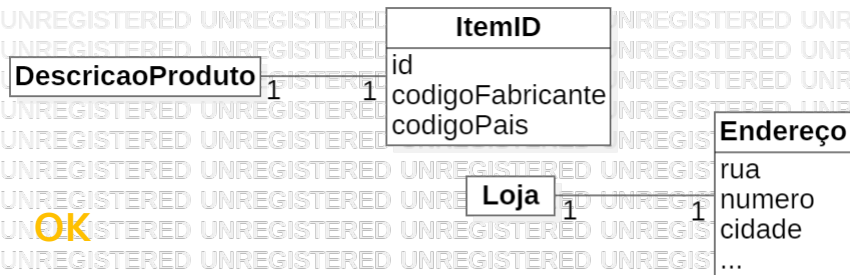
Leonor Melo

o9 Modelo do Domínio

12

12

Modelo do domínio:
representar
classes tipos
de dados? -
notação UML -
2



DescricaoProduto

idItem: ItemID

OK

Loja

localizacao: Endereço

- ItemID e Endereço são classes tipo de dados
 - para as comparar as suas instancias usamos o seu valor e não a sua identidade

Classes
abstratas

- Aplica-se a
 - classes mais genéricas
 - desenhadas para serem reutilizadas
 - agregam atributos e comportamentos comuns às subclasses
- Algum do comportamento da classe mais genérica é desconhecido
 - Sei que o comportamento deve existir
 - Mas implementação depende exclusivamente da classe derivada

Classes abstratas

Repositorio

+guarda(artigos: Artigo[*]): void
+recupera(): Artigo[*]

- Métodos abstratos
 - assinatura escrita em itálico
 - não contém implementação
 - "espaço reservado"
 - a implementação será feita pelas subclasses

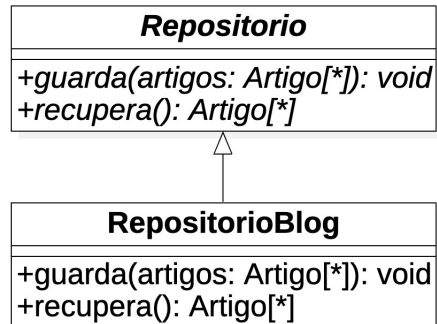
Classes abstratas

Repositorio

+guarda(artigos: Artigo[*]): void
+recupera(): Artigo[*]

- Se algum dos métodos for abstrato
 - a classe é abstrata
- Classe abstrata
 - Não pode ser instanciada

Classes abstratas



- Subclasse de classe abstrata
 - Ou implementa os todos os métodos abstratos
 - Ou é também abstrata

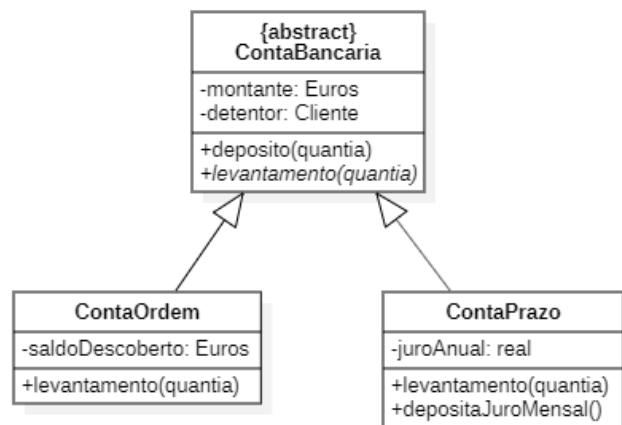
Leonor Melo

15 - Diagramas de classe

17

17

Classes abstratas



- normalmente
 - tem atributos
 - apenas parte das operações são abstratas

Leonor Melo

15 - Diagramas de classe

18

18

Classes abstratas

- Classes abstratas
 - Mecanismo que promove a reutilização
 - Define-se atributos e operações que deverão existir / ser garantidos
 - Ainda que os detalhes venham a ser definidos pelas sub- classes
 - Usadas frequentemente em design patterns
 - Usa relação de herança
 - Acoplamento forte
 - Usar com cuidado

Leonor Melo

15 - Diagramas de classe

19

19

Interfaces

- Representa um tipo de comportamento que determinada classe poderá, ou não, ser capaz de fornecer
- Conjunto de operações
 - sem implementação dos métodos
- "Contrato" que indica
 - a assinatura das operações que devem existir
- Raramente contém atributos
 - e apenas estáticos ou constantes
- Não pode ser instanciada

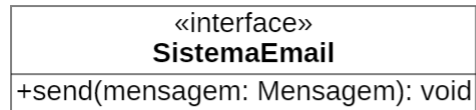
Leonor Melo

15 - Diagramas de classe

20

20

Notação UML



Notação com esteriótipo

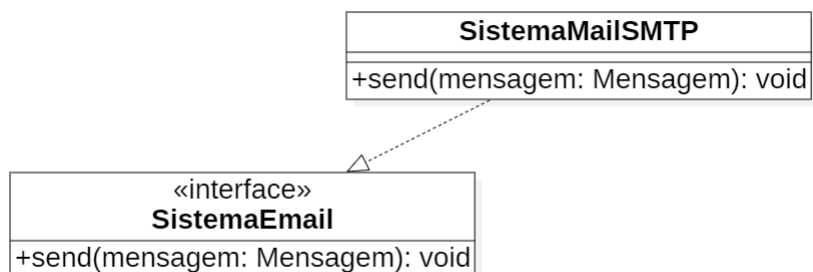


Notação com "bola"

21

Relação de realização

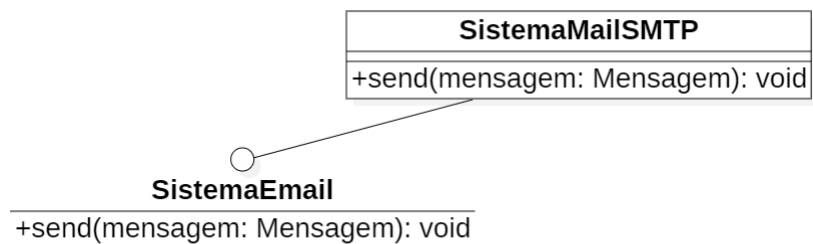
- “relação de realização”
 - Estabelecida entre uma classe e um interface
 - A classe está em conformidade com o contrato especificado pela interface



22

Relação de realização

- representação UML
 - na notação de bola
 - linha de associação
- Notação com esteriótipos
 - seta de realização



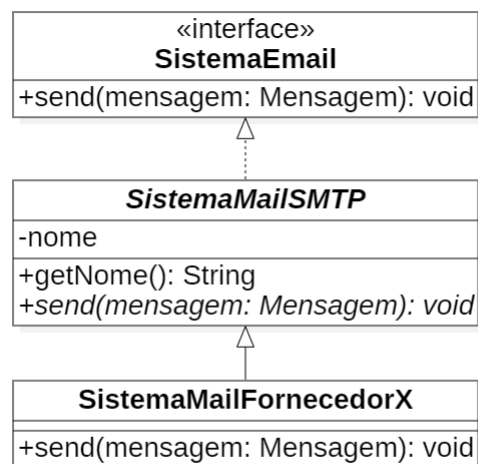
Leonor Melo

15 - Diagramas de classe

23

23

Relação de realização



- Uma classe que realize um interface mas não implemente todos os métodos do interface tem de ser declarada abstrata

Leonor Melo

15 - Diagramas de classe

24

24

Interface: relação de dependencia

- Os interfaces existem para que as classes dependam dos interfaces e não de classes específicas (abstração, redução de acoplamento)
- Uma classe A depende de um interface se
 - é indiferente os detalhes da classe que na realidade vai executar os métodos
 - a classe A sabe que a classe que eventualmente implementar o interface está preparada para receber mensagens com determinada assinatura

Leonor Melo

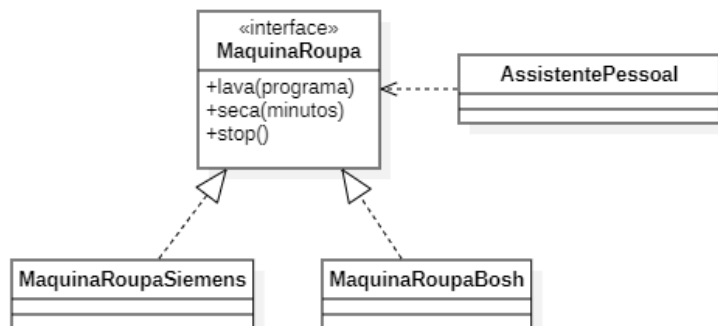
15 - Diagramas de classe

25

25

Interface: relação de dependencia

- O assistente sabe como comunicar com uma máquina da roupa, desde que essa máquina realize o interface MaquinaRoupa



Leonor Melo

15 - Diagramas de classe

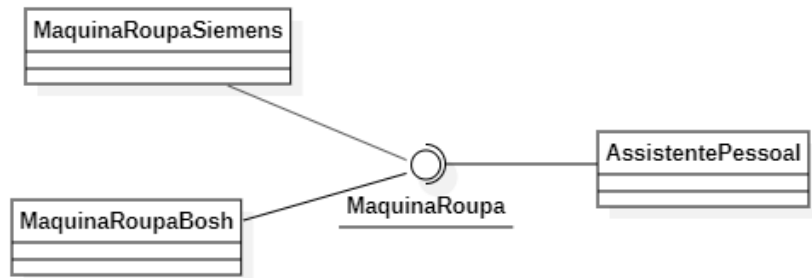
26

26

Interface: relação de dependência

- Notação de bola:

- Tanto pode comunicar com uma máquina Bosh como com uma Siemens (ou outras que entretanto realizem o interface)



Leonor Melo

15 - Diagramas de classe

27

27

Interfaces

- Uma classe pode realizar vários interfaces
- Evita problemas da generalização múltipla
 - porquê?
- Semelhantes a classes abstratas
 - mas com acoplamento mais fraco
 - usadas para "desacoplar" classes
- Usadas frequentemente em design patterns

Leonor Melo

15 - Diagramas de classe

28

28

Interfaces

- Separar o comportamento que a classe deve ter
 - da implementação desse comportamento
- Se uma classe implementar um interface
 - objetos dessa classe podem ser referidos pelo nome do interface
- As outras classes podem passar a depender do interface
 - e não da classe que o implementa

Leonor Melo

15 - Diagramas de classe

29

29

Templates

- classe parameterizada
 - útil quando se quer adiar a decisão sobre com que classe(s) a nossa classe irá trabalhar
- sei que a minha classe irá trabalhar com outras
 - mas não quero ter de especificar exatamente com qual

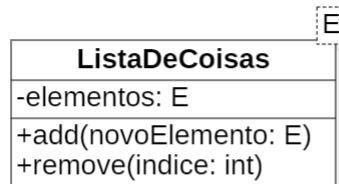
Leonor Melo

15 - Diagramas de classe

30

30

Templates



- E não é nenhuma classe existente no modelo
 - está apenas no lugar onde aparecerá a classe cujos objetos serão guardados na lista

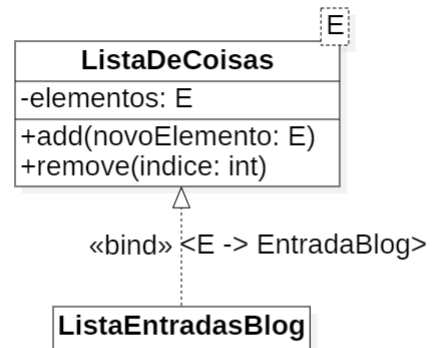
31

Usar Templates

- usar uma classe template
 - fazer o *bind* (ligar) dos parâmetros
- Fazer o *bind*:
 - *binding* por subclasse
 - *binding* em *runtime*

32

Usar Templates: *binding* por subclasse



- ListaEntradasBlog:
 - comportamento genérico de ListaDeCoisas
 - apenas para instancias de EntradaBlog

33

Usar Templates: *binding* em *runtime*

- O *template* só conhece o tipo de dados com que vai trabalhar quando é criado
 - em *runtime*
- *Binding* que diz respeito aos objetos
 - representado usando diagramas de objeto

34